1. Тема работы: "Проектирование архитектуры системы: разработка UML-диаграммы классов и схемы базы данных"

2. Цель работы:

Сформировать практические навыки проектирования программных систем:

1. Научиться проектировать высокоуровневую логическую структуру системы с помощью UML-диаграммы классов
2. Научиться трансформировать логическую модель в физическую схему базы данных
3. Получить опыт обоснования принятых архитектурных решений

3. Теоретическая справка

3.1. UML-диаграмма классов

Статическая структурная диаграмма, описывающая:

- Классы системы, их атрибуты и методы
- Отношения между классами (ассоциация, наследование, агрегация, композиция)
- Является основой для написания кода

3.2. Схема базы данных

Логическая структура БД, включающая:

- Таблицы с полями и типами данных
- Первичные и внешние ключи
- Ограничения (NOT NULL, UNIQUE)
- Является физической реализацией модели данных

4. Задание на лабораторную работу

Этап 1: Разработка UML-диаграммы классов

Задача: Создать UML-диаграмму классов, отражающую ключевые сущности системы.

Процесс:

В рамках лабораторной работы была выполнена декомпозиция архитектуры программного комплекса, предназначенного для создания, редактирования и публикации карт настольных ролевых игр. Архитектура разделена на две части: клиентскую, реализуемую средствами JavaScript и отображаемую в виде набора модулей с префиксом DM, и серверную, реализуемую на платформе Flask, выполняющую роль веб-сервера и обслуживающую галерею карт с комментариями. Разделение зафиксировано в двух

диаграммах классов, описывающих соответственно фронтенд и бэкенд системы. Полные диаграммы находится в приложении А.

3.1. Frontend часть

Фронтенд-часть организована вокруг корневого пространства имён DM, которое выступает единой точкой входа. В состав корневого модуля входят четырнадцать подсистем, каждая из которых решает свою задачу и обособлена в отдельный модуль. Конфигурация системы вынесена в замороженный объект DM_Config, содержащий размеры карты MAP_W и MAP_H, толщину стен WALL_T, а также наборы инструментов для областей и точек, типы покрытий, границ и их значения по умолчанию, группы инструментов, приоритеты стирания и цветовую палитру. Такое решение позволяет централизованно управлять всеми константами и не дублировать их в коде.

Центральным элементом клиентской части является модуль состояния DM_State, который хранит текущее состояние документа: рабочую копию данных, размеры карты, параметры сетки, масштаб, активный инструмент, массив слоёв с индексом активного слоя, массив элементов карты, массив пользовательских ассетов, историю изменений и стек отмены, а также информацию о выделении — одиночном и множественном. Модуль предоставляет методы управления слоями, включая добавление, удаление, перемещение, переименование, переключение видимости и блокировки, методы работы с историей — saveHistory, undo и redo, методы работы с элементами — добавление, удаление, обновление, а также методы сериализации и десериализации всего состояния. Отдельная группа методов отвечает за работу с ассетами. Внутренние вспомогательные методы отвечают за создание слоя, снятие снимка слоёв и восстановление слоёв из снимка.

Генерация случайных значений вынесена в модуль DM_RNG, представляющий собой обёртку над генератором псевдослучайных чисел с детерминированным сидом, и функцию DM_cellRNG, возвращающую отдельный генератор для каждой клетки карты, что обеспечивает воспроизводимость процедурной генерации. Модуль рисования DM_Draw инкапсулирует всю работу с двумерной графикой: отрисовку фона, сетки, полов, стен, водных и ямных элементов, границ комнат, секретных дверей и лестниц, изображений, текста, настенных элементов, кистей и курсоров, тумана, декалей, источников света, выделения и предпросмотра. Внутри модуля реализованы служебные функции для переноса строк текста, разбиения подпутей и разбора HEX-цветов. С модулем рисования тесно связаны DM_BrushPatterns, отвечающий за кэширование и получение шаблонов кистей, и DM_Utils, предоставляющий утилиты для работы с анимационными кадрами, прямоугольниками и центрами элементов, привязкой к сетке, построением сетки клеток и сегментов стен.

Рендеринг карты и управление масштабом реализованы в модулях DM_Renderer и DM_Zoom. Первый владеет основным и вспомогательными канвасами, кэширует сетку и версии карты, выполняет отрисовку, экспорт в высоком разрешении и анимацию декалей. Второй инкапсулирует минимальный и максимальный масштаб, дискретные шаги и операции применения, установки, приближения, отдаления и сброса масштаба. Инструментальная подсистема DM_Tools отвечает за обработку пользовательского ввода: нажатий, перемещений и отпусканий мыши, горячих клавиш, стирания, проверки попадания в элемент, поиска верхнего кандидата, изменения размеров, поворота, копирования, вставки, дублирования, слияния мазков кисти и штампов тумана, а также за панорамирование и выделение рамкой. Генератор карт DM_Generator на основе алгоритма BSP создаёт случайные комнаты с настраиваемым количеством, минимальным размером, долей круглых комнат и шагом колонн.

Ввод-вывод сосредоточен в модуле DM_IO, который кэширует изображения, хранит ключи сессии и облака, реализует сохранение и загрузку карты, экспорт в PNG и JPG, загрузку изображений на сервер, получение списка серверных ассетов, удаление ассетов, автосохранение и восстановление сессии. Пользовательский интерфейс вынесен в DM_UI, который инициализирует все панели — сетку, фон, размер карты, генератор, ассеты, комнаты, кисти, туман, двери, заливку, зум, кнопки шапки и боковую панель, обновляет свойства, мультивыделение, список ассетов и панель слоёв, управляет метаданными ассетов и пользовательскими категориями, синхронизирует элементы управления и открывает модальные окна, включая окно галереи. Аутентификация возложена на DM_Auth, который хранит текущего пользователя, CSRF-токен и шину событий, выполняет вход, регистрацию и выход, показывает и скрывает модальное окно, создаёт виджет в шапке и обновляет его.

Модуль DM_Cloud работает с картами, ассетами, ссылками для игры и комментариями, реализует менеджеры карт и ассетов.

Модуль публикации DM_Share формирует бинарный объект изображения, собирает теги, получает подсказки, строит ссылки для платформ и обновляет кнопки публикации.

Модуль галереи DM_Gallery, обеспечивает открытие карты, загрузку, добавление, редактирование и удаление комментариев, установку режима сортировки и подписку на живые обновления, а также проверку прав на редактирование и удаление.

Связи фронтенд-части построены по двум принципам. Отношения композиции DM *-- X показывают, что корневое пространство имён включает в себя все перечисленные модули как составные части. Отношения зависимости ..> отражают реальное использование: DM_Draw зависит от DM_BrushPatterns и DM_Utils, DM_Generator — от DM_RNG, DM_Renderer — от DM_Draw и DM_Utils, DM_Tools — от DM_Utils, DM_UI — от DM_Generator, DM_Cloud — от DM_IO, DM_Auth и DM_UI, DM_Share — от DM_IO и DM_Auth, DM_Gallery — от DM_Auth и DM_UI, а DM_UI, в свою очередь, вызывает DM_Gallery. Последние две зависимости образуют цикл между интерфейсом и галереей, что отражает двустороннее взаимодействие: интерфейс открывает галерею, а галерея после действий пользователя возвращает управление в интерфейс.

3.1. Backend часть

Бэкенд-часть диаграммы описывает серверную реализацию, а именно галереи карт и комментариев. Ядром серверной части является класс App, построенный по шаблону фабрики приложений Flask. Он владеет конфигурацией, объектом SQLAlchemy, чертежом галереи и методами создания приложения, регистрации чертежей и внутренней инициализации базы данных и обработчиков. Конфигурация вынесена в неизменяемый объект Config, содержащий строку подключения к базе данных, секретный ключ, максимальную длину комментария, размер страницы галереи и список разрешённых источников CORS. Маршруты галереи собраны в GalleryBlueprint с префиксом /api/gallery, который предоставляет получение списка карт с сортировкой, получение одной карты, получение комментариев карты, создание комментария, редактирование и удаление комментария, а также поток событий через Server-Sent Events и внутренний метод получения текущего пользователя.

Данные хранятся в трёх ORM-моделях SQLAlchemy. Пользователь User содержит идентификатор, имя, хэш пароля, признак активности и коллекцию комментариев, а также методы проверки пароля и преобразования в публичное представление. Карта Map содержит идентификатор, идентификатор владельца, имя, JSON-сериализацию данных карты, ссылку на превью, признак публичности, дату создания и метод преобразования в публичное представление. Комментарий Comment содержит идентификатор, идентификатор карты, идентификатор автора, текст, даты создания и обновления, а также методы проверки прав на редактирование и удаление. Схемы обмена данными вынесены в модуль DTOs на базе marshmallow и включают публичное представление пользователя, публичное представление карты, схемы создания и вывода комментария, а также схемы списка галереи и режима сортировки.

Доступ к данным организован через три репозитория. MapRepo предоставляет выборку публичных карт с постраничной разбивкой и сортировкой и выборку публичной карты по идентификатору. CommentRepo отвечает за получение комментария, список комментариев карты, создание, обновление и удаление. UserRepo предоставляет получение пользователя по идентификатору и по имени. Бизнес-логика сосредоточена в GalleryService, который реализует получение списка карт, открытие карты, получение списка комментариев, добавление, редактирование и удаление комментария, а также подписку на события карты и внутренние проверки прав на редактирование и удаление. Доставка живых обновлений возложена на SubscriptionHub, который предоставляет подписку, отписку и широковещательную рассылку событий в рамках карты. Инфраструктурные модули Database и Middleware отвечают соответственно за инициализацию SQLAlchemy и за обработку запросов до и после их выполнения, включая проверку аутентификации, CSRF-защиту и обработку ошибок.

Связи бэкенд-части отражают слоистую архитектуру. App владеет Config, Database, Middleware и GalleryBlueprint, то есть включает их как составные части. GalleryBlueprint зависит от GalleryService, который, в свою очередь, зависит от трёх репозиториев — MapRepo, CommentRepo, UserRepo — и от SubscriptionHub. Модель Comment связана с Map отношением принадлежности и с User отношением авторства, пользователь пишет много комментариев, карта имеет много комментариев, что зафиксировано отношениями один-к-одному и один-ко-многим соответственно. Такое разделение обеспечивает изоляцию уровней: маршруты не знают о базе данных, сервисы не знают о HTTP, а модели описывают только структуру данных и правила доступа.

Этап 3: Проектирование схемы базы данных

Задача: Создать схему реляционной базы данных на основе UML-диаграммы.

Процесс:

1. Трансформируйте UML-диаграмму в схему БД по правилам:
   - Класс → Таблица
   - Атрибут → Поле таблицы
   - Типы атрибутов → SQL типы данных
2. Для каждой таблицы укажите:
   - Первичный ключ (id, автоинкремент)
   - Внешние ключи для связей
   - Ограничения (NOT NULL, UNIQUE)
3. Решите вопрос наследования (единая таблица, таблица на класс)
4. Используйте инструменты: draw.io, dbdiagram.io, MySQL Workbench

Результат: FA-диаграмма или документ с описанием таблиц

Приложение А

Приложение А — Диаграммы классов

В приложении приведены диаграммы классов системы «Dungeon Map Builder»: общая диаграмма классов фронтенда и её разбиение на слои (поддиаграммы), а также диаграмма классов бэкенда (Flask) с разбиением на слои. Каждая диаграмма сопровождается ссылкой на исходный файл в формате Mermaid.

A.1. Фронтенд

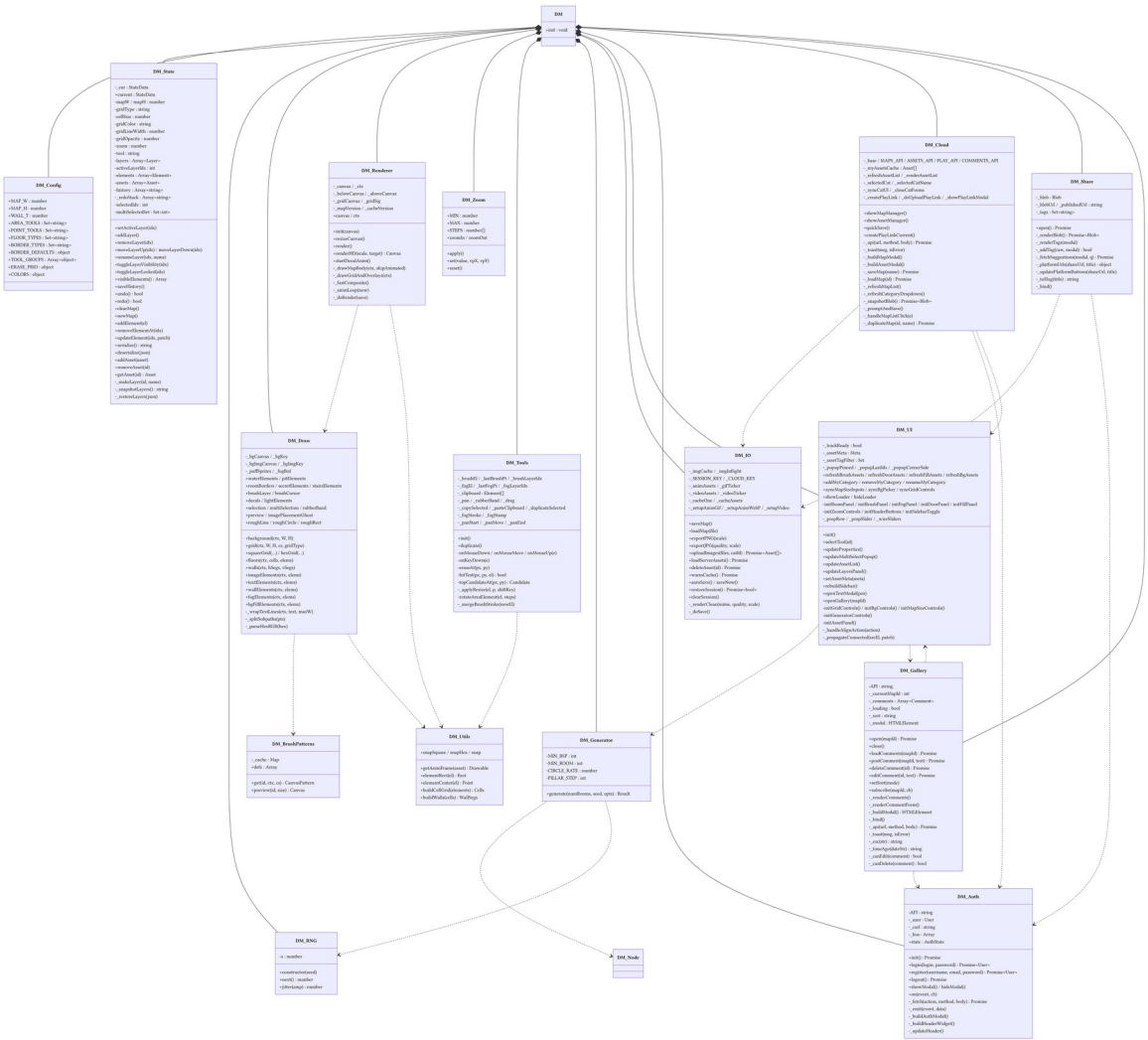


Рис. 1 — Диаграмма классов фронтенда (общая)

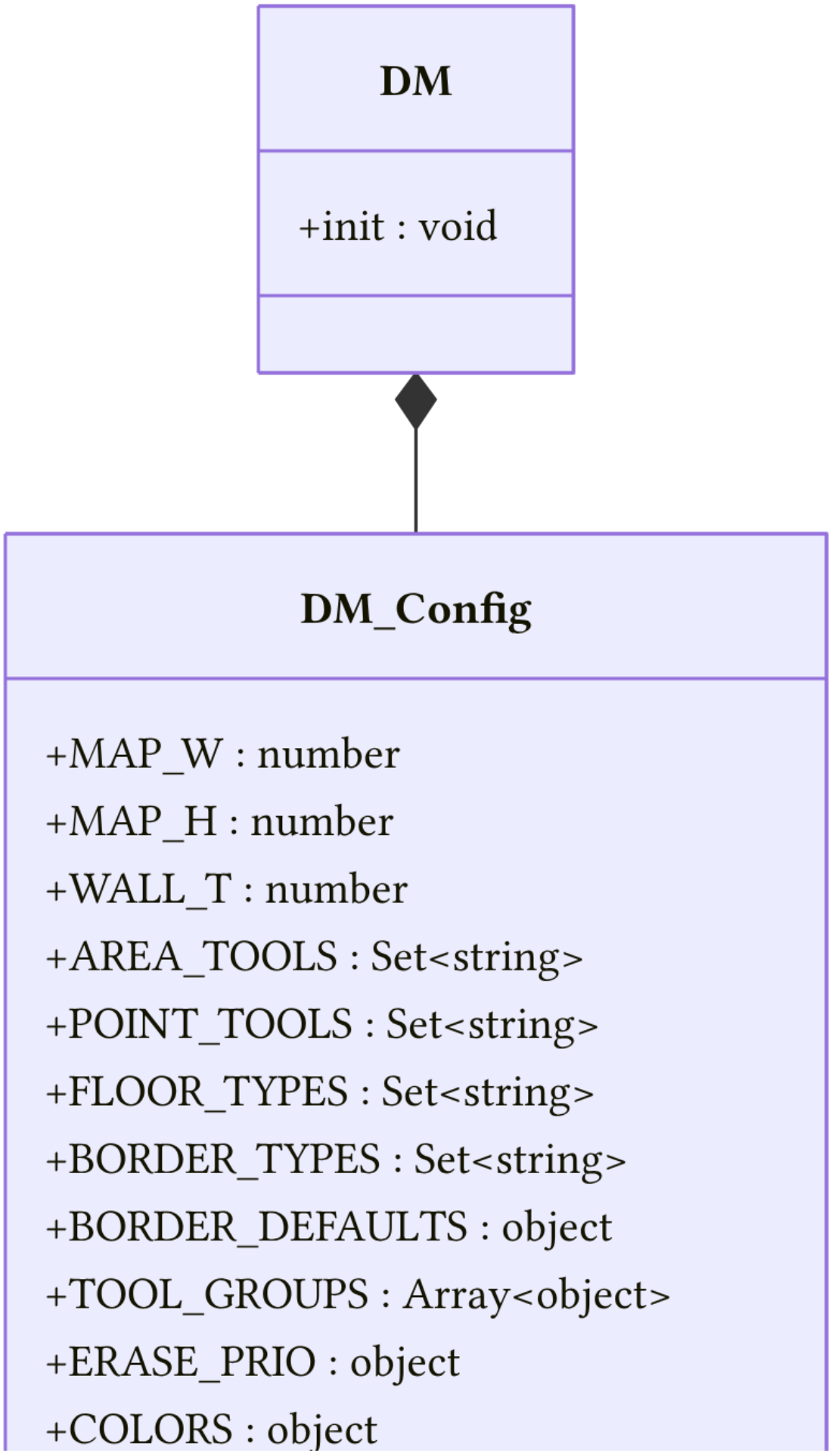

DM
+init : void
DM_Config
+MAP_W : number
+MAP_H : number
+WALL_T : number
+AREA_TOOLS : Set<string>
+POINT_TOOLS : Set<string>
+FLOOR_TYPES : Set<string>
+BORDER_TYPES : Set<string>
+BORDER_DEFAULTS : object
+TOOL_GROUPS : Array<object>
+ERASE_PRIO : object
+COLORS : object

Рис. 2 — Слой 1 — Ядро и конфигурация

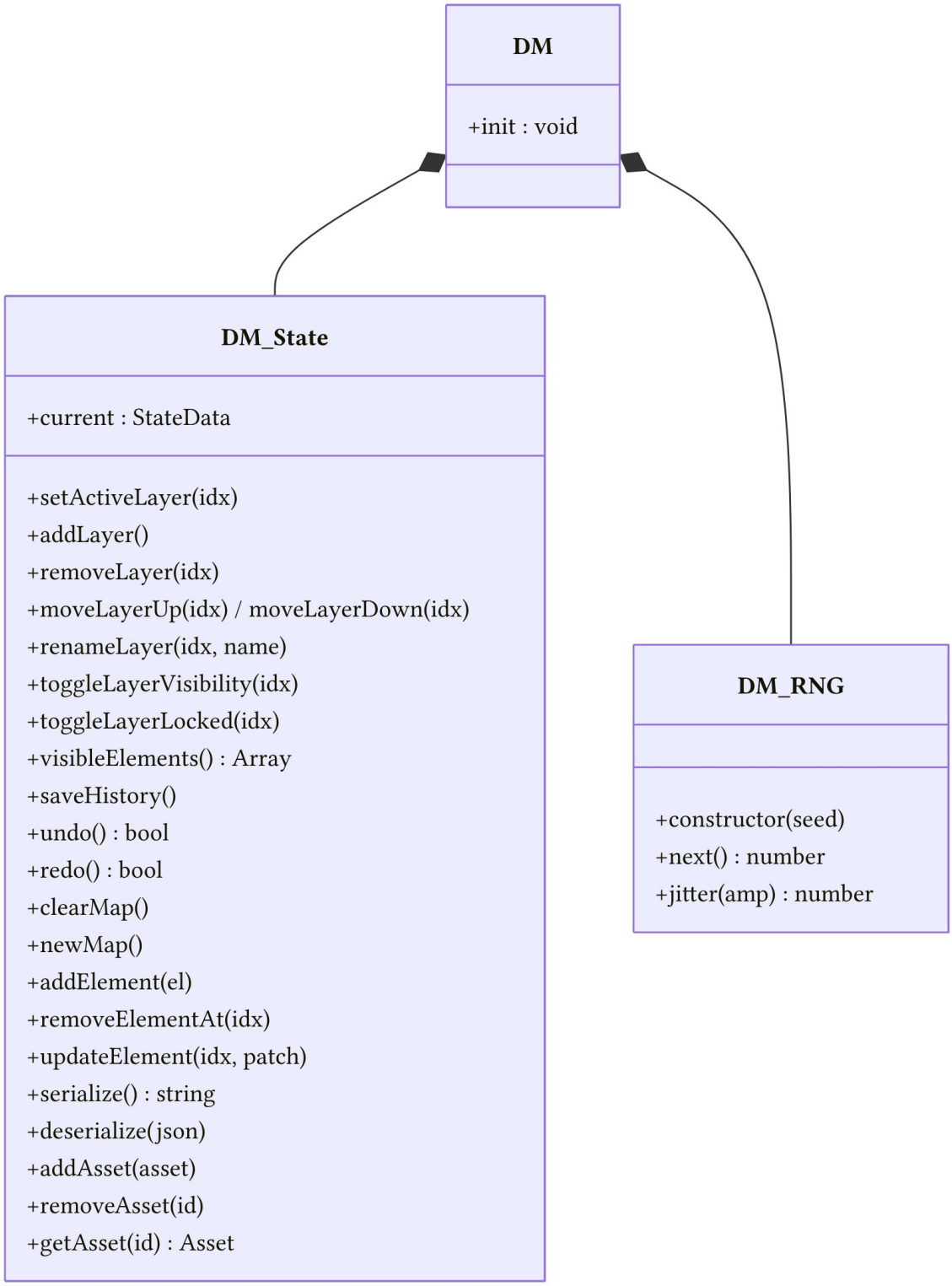


Рис. 3 — Слой 2 — Состояние и случайность

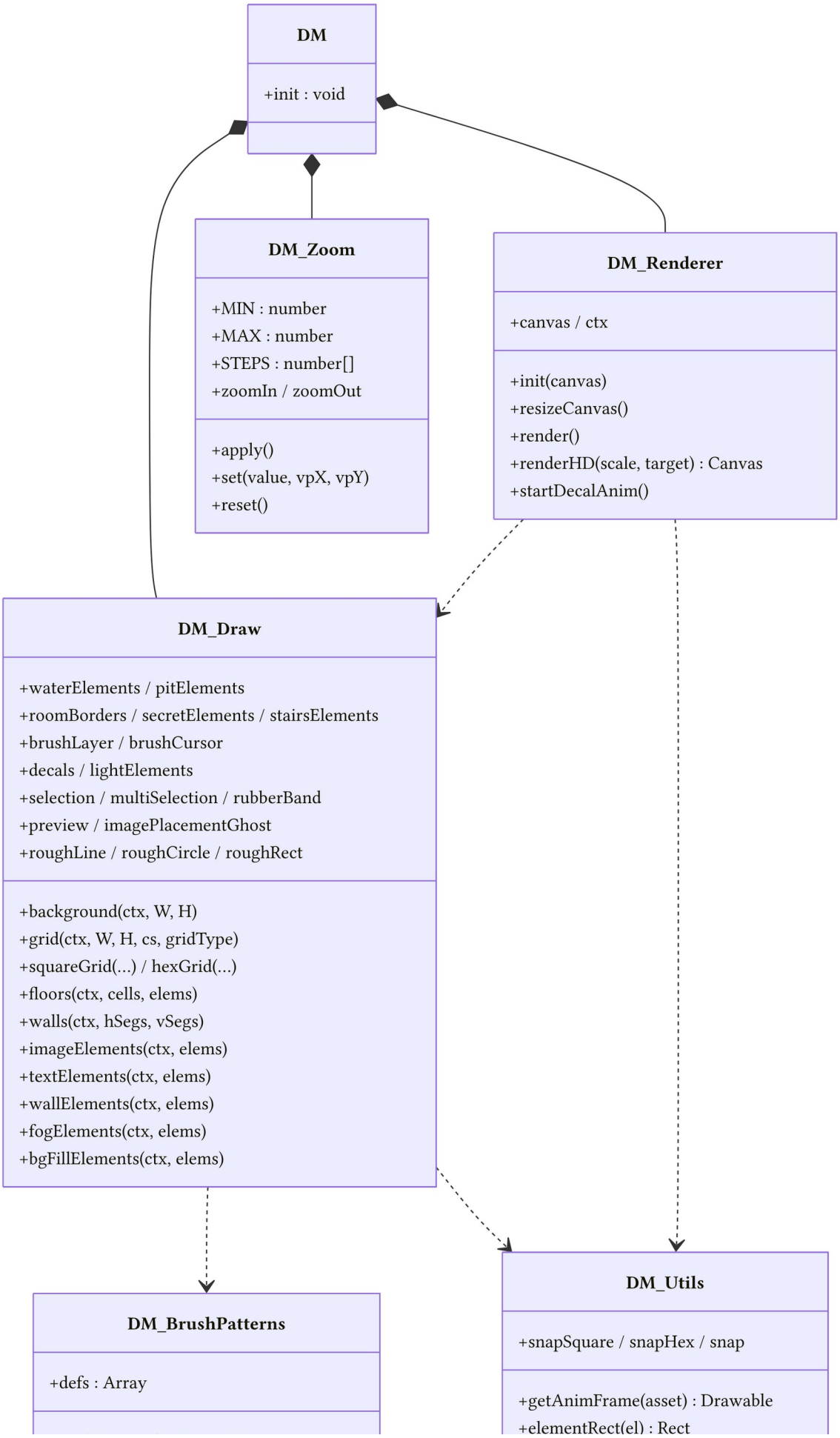

DM
+init : void
DM_Zoom
+MIN : number
+MAX : number
+STEPS : number[]
+zoomIn / zoomOut
+apply()
+set(value, vpX, vpY)
+reset()
DM_Renderer
+canvas / ctx
+init(canvas)
+resizeCanvas()
+render()
+renderHD(scale, target) : Canvas
+startDecalAnim()
DM_Draw
+waterElements / pitElements
+roomBorders / secretElements / stairsElements
+brushLayer / brushCursor
+decals / lightElements
+selection / multiSelection / rubberBand
+preview / imagePlacementGhost
+roughLine / roughCircle / roughRect
+background(ctx, W, H)
+grid(ctx, W, H, cs, gridType)
+squareGrid(...) / hexGrid(...)
+floors(ctx, cells, elems)
+walls(ctx, hSegs, vSegs)
+imageElements(ctx, elems)
+textElements(ctx, elems)
+wallElements(ctx, elems)
+fogElements(ctx, elems)
+bgFillElements(ctx, elems)
DM_BrushPatterns
+defs : Array
DM_Utils
+snapSquare / snapHex / snap
+getAnimFrame(asset) : Drawable
+elementRect(el) : Rect

Рис. 4 — Слой 3 — Отрисовка и рендеринг

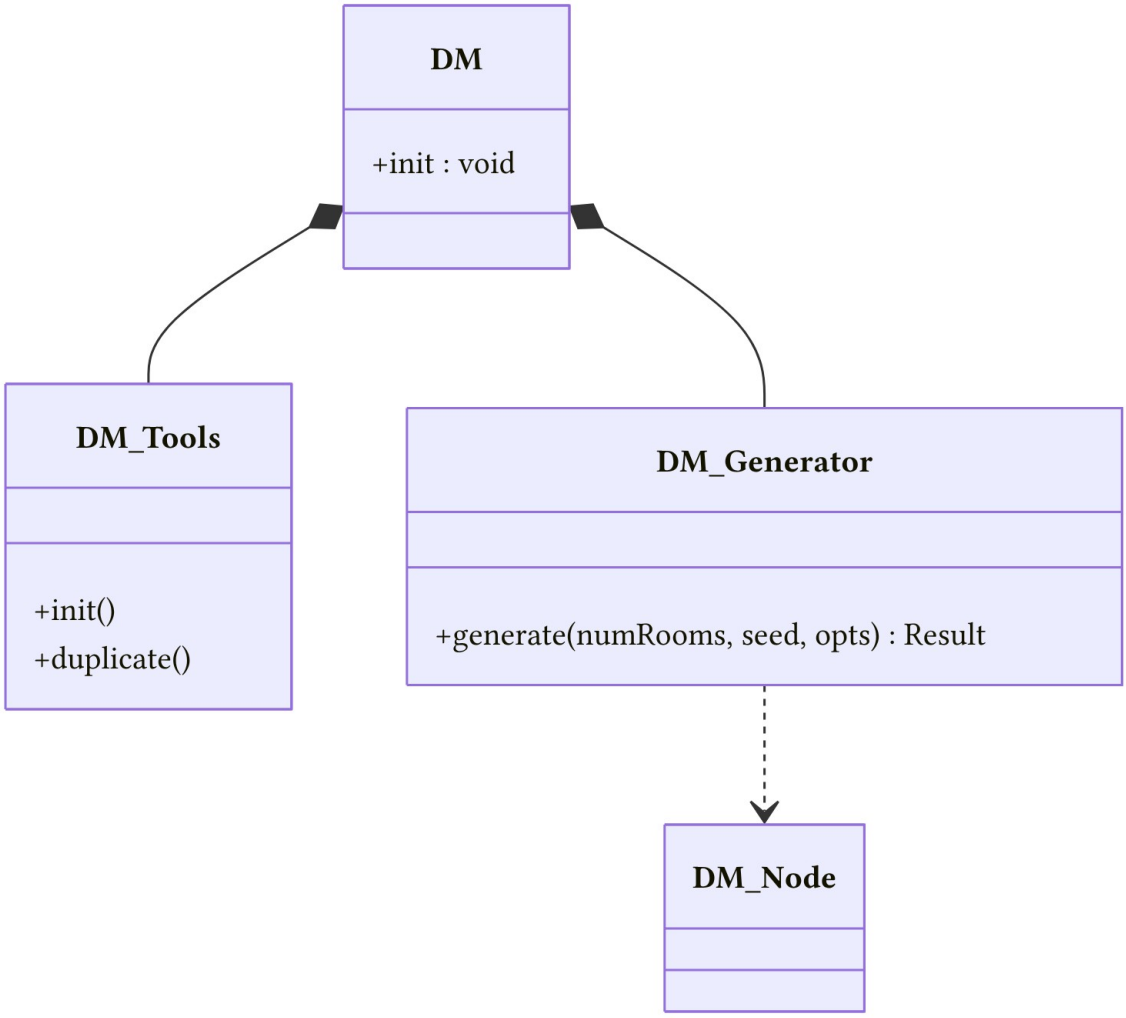


Рис. 5 — Слой 4 — Инструменты и генератор

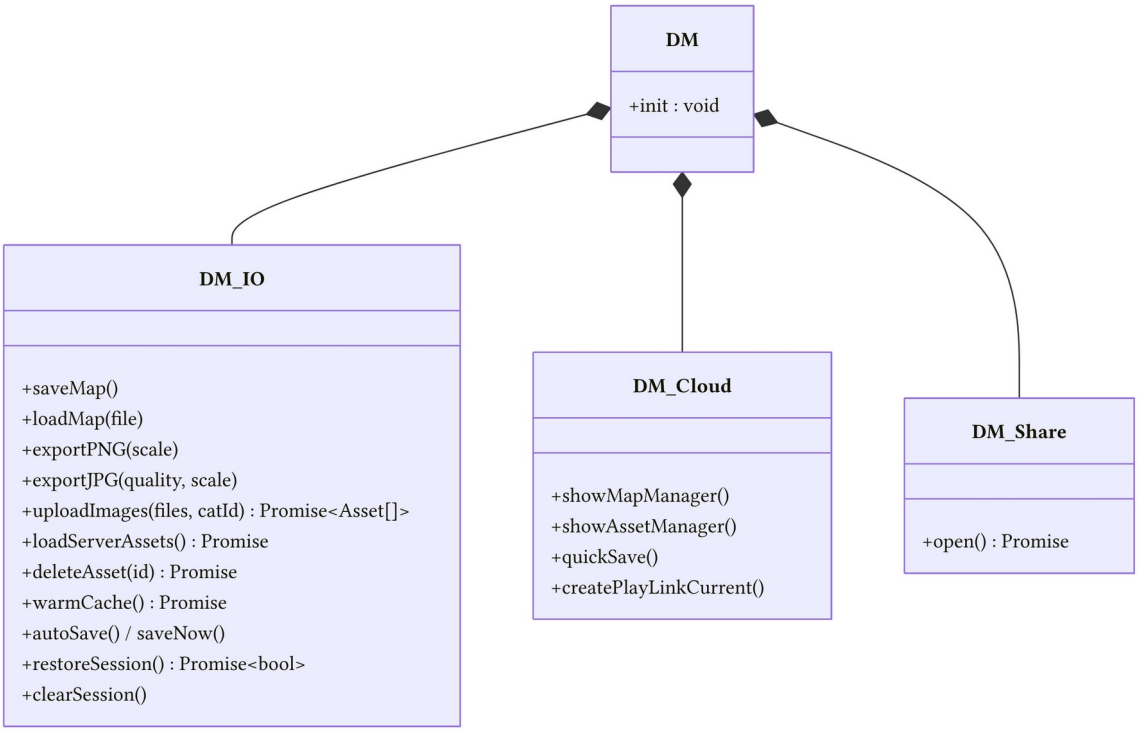


Рис. 6 — Слой 5 — IO, облако и публикация

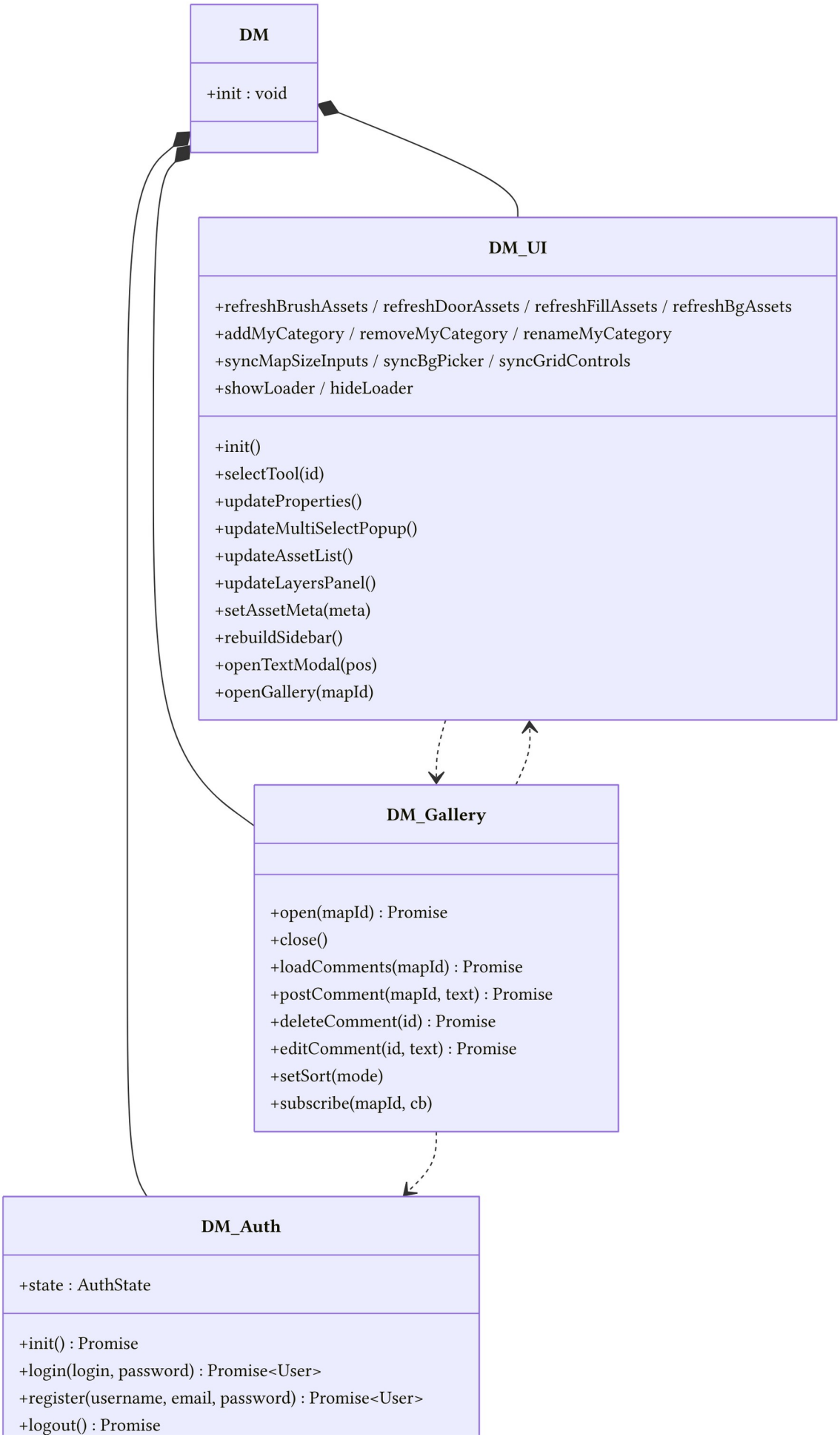

DM
+init : void
DM_UI
+refreshBrushAssets / refreshDoorAssets / refreshFillAssets / refreshBgAssets
+addMyCategory / removeMyCategory / renameMyCategory
+syncMapSizeInputs / syncBgPicker / syncGridControls
+showLoader / hideLoader
+init()
+selectTool(id)
+updateProperties()
+updateMultiSelectPopup()
+updateAssetList()
+updateLayersPanel()
+setAssetMeta(meta)
+rebuildSidebar()
+openTextModal(pos)
+openGallery(mapId)
DM_Gallery
+open(mapId) : Promise
+close()
+loadComments(mapId) : Promise
+postComment(mapId, text) : Promise
+deleteComment(id) : Promise
+editComment(id, text) : Promise
+setSort(mode)
+subscribe(mapId, cb)
DM_Auth
+state : AuthState
+init() : Promise
+login(login, password) : Promise<User>
+register(username, email, password) : Promise<User>
+logout() : Promise

Рис. 7 — Слой 6 — UI, авторизация и галерея

A.2. Бэкенд

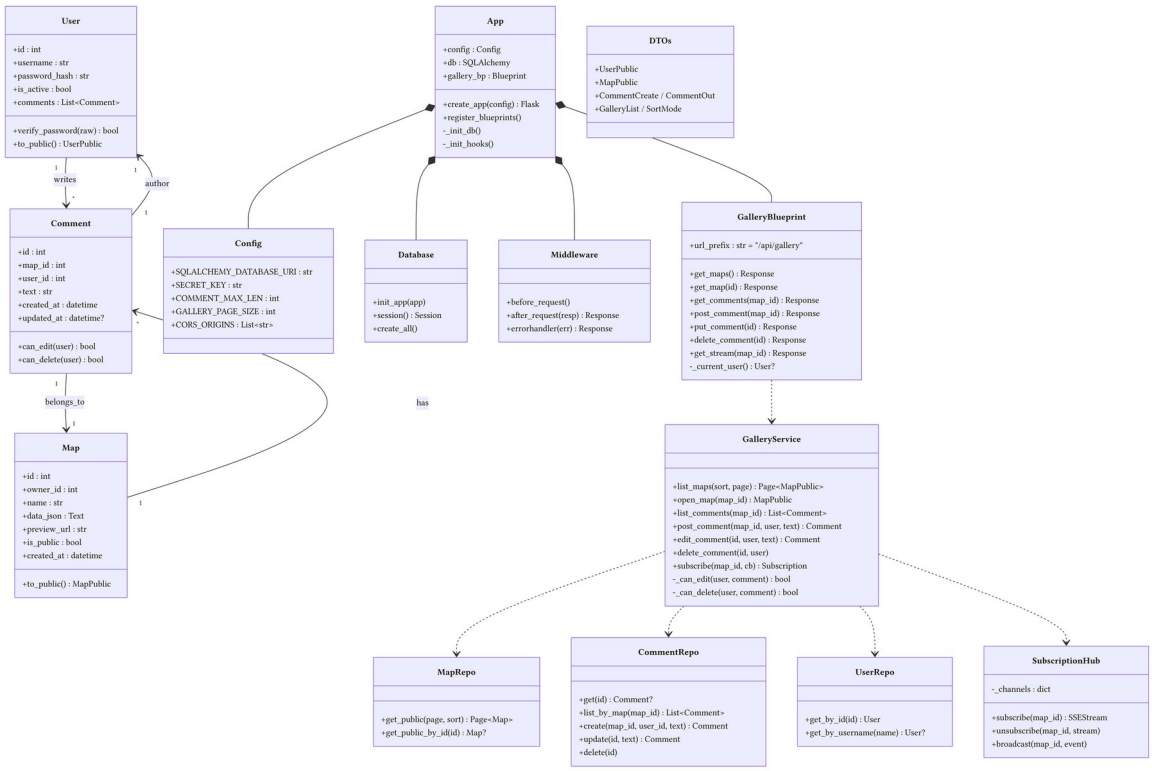


Рис. 8 — Диаграмма классов бэкенда (общая)

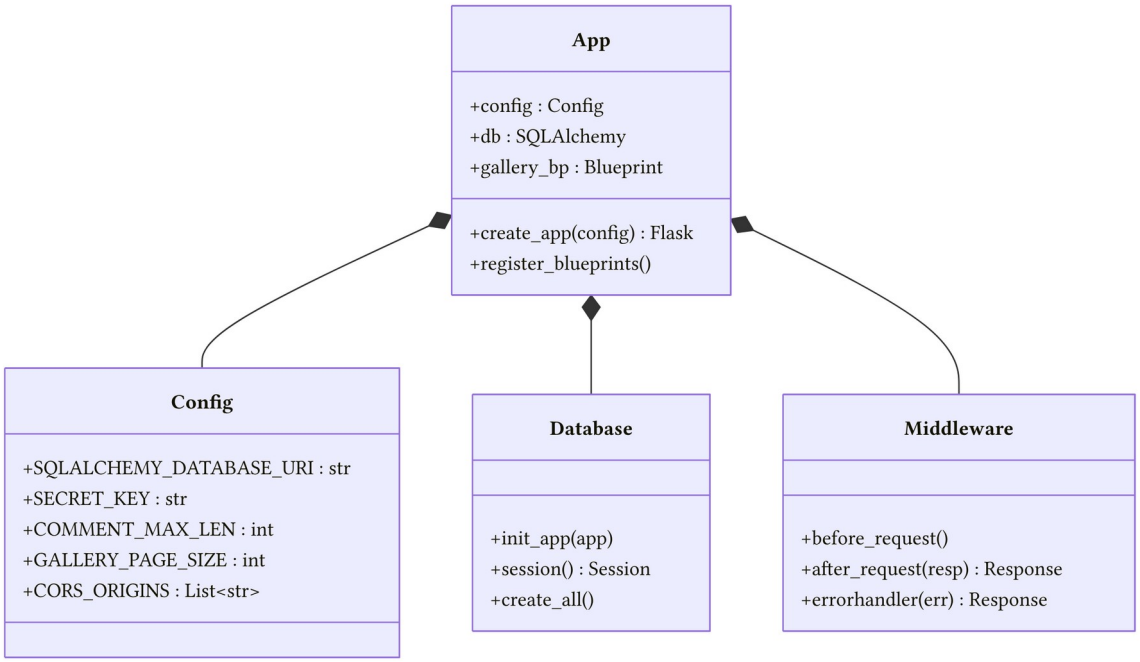


Рис. 9 — Слой 1 — Каркас приложения

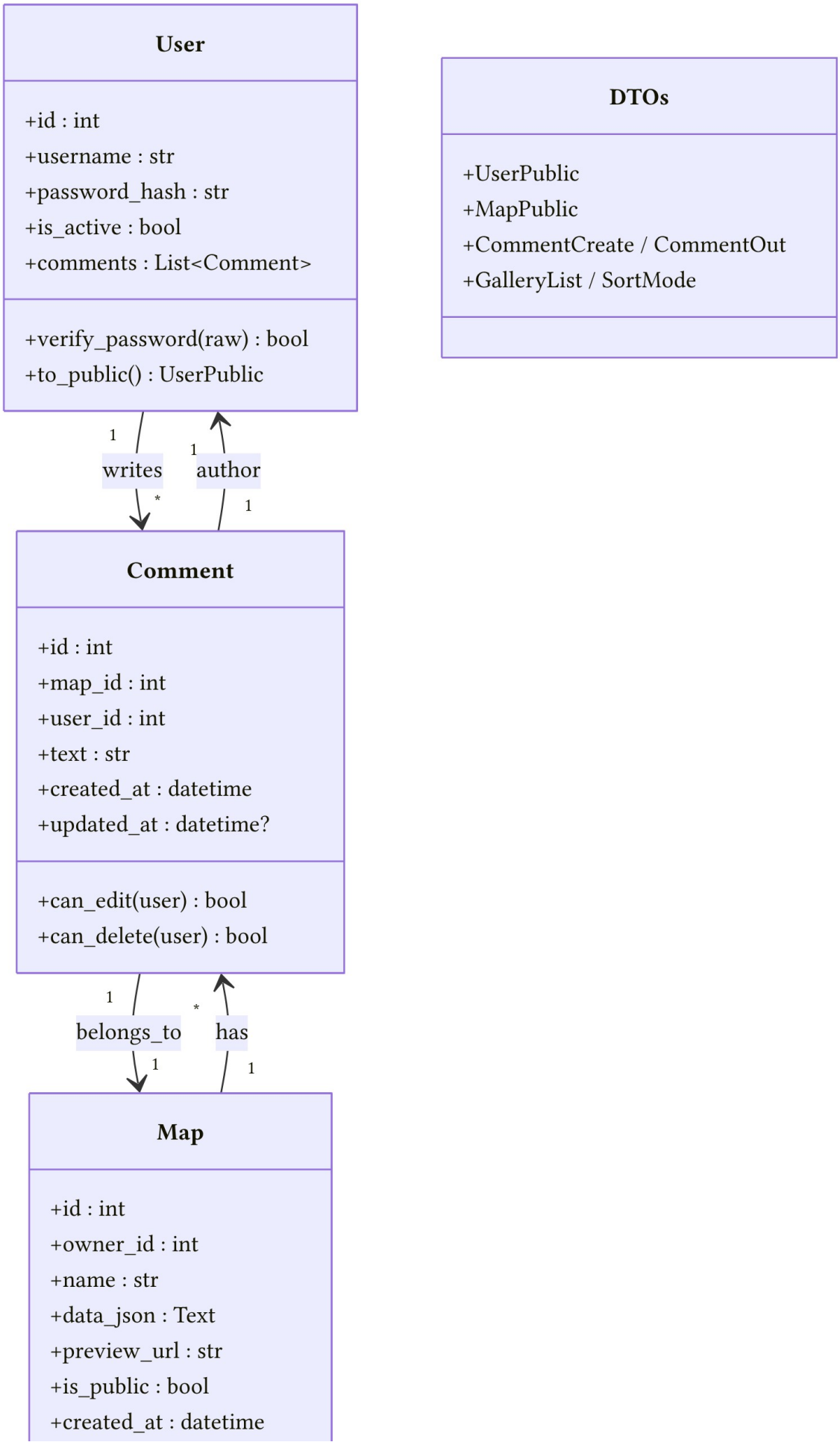

User
+id : int
+username : str
+password_hash : str
+is_active : bool
+comments : List<Comment>
+verify_password(raw) : bool
+to_public() : UserPublic
DTOs
+UserPublic
+MapPublic
+CommentCreate / CommentOut
+GalleryList / SortMode
1
writes
*
1
author
1
Comment
+id : int
+map_id : int
+user_id : int
+text : str
+created_at : datetime
+updated_at : datetime?
+can_edit(user) : bool
+can_delete(user) : bool
1
belongs_to
1
*
has
1
Map
+id : int
+owner_id : int
+name : str
+data_json : Text
+preview_url : str
+is_public : bool
+created_at : datetime

Рис. 10 — Слой 2 — Модели данных

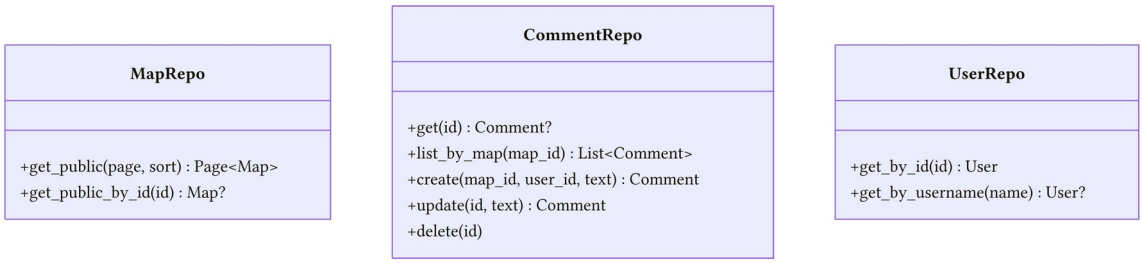


Рис. 11 — Слой 3 — Репозитории

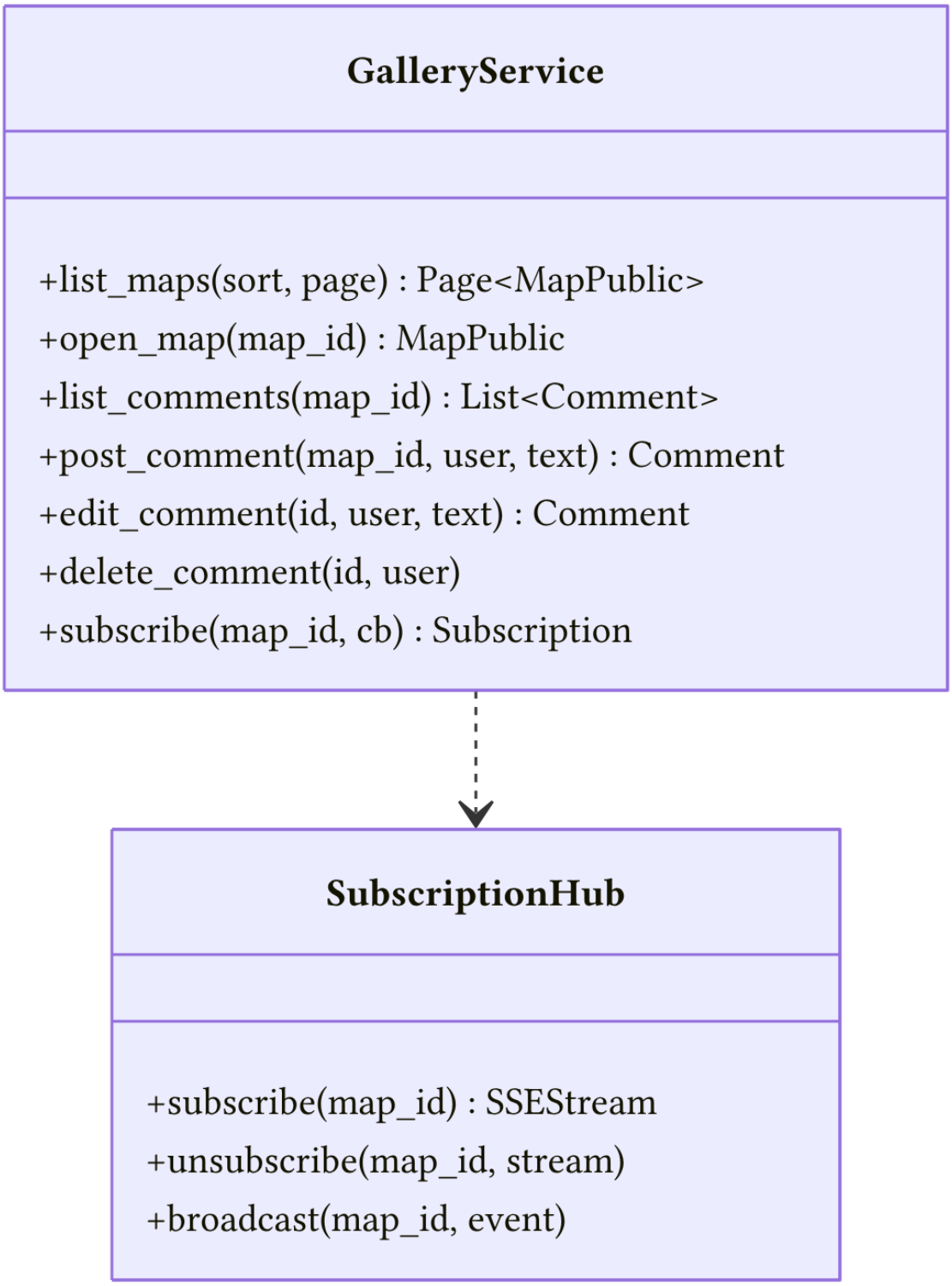


Рис. 12 — Слой 4 — Сервис галереи

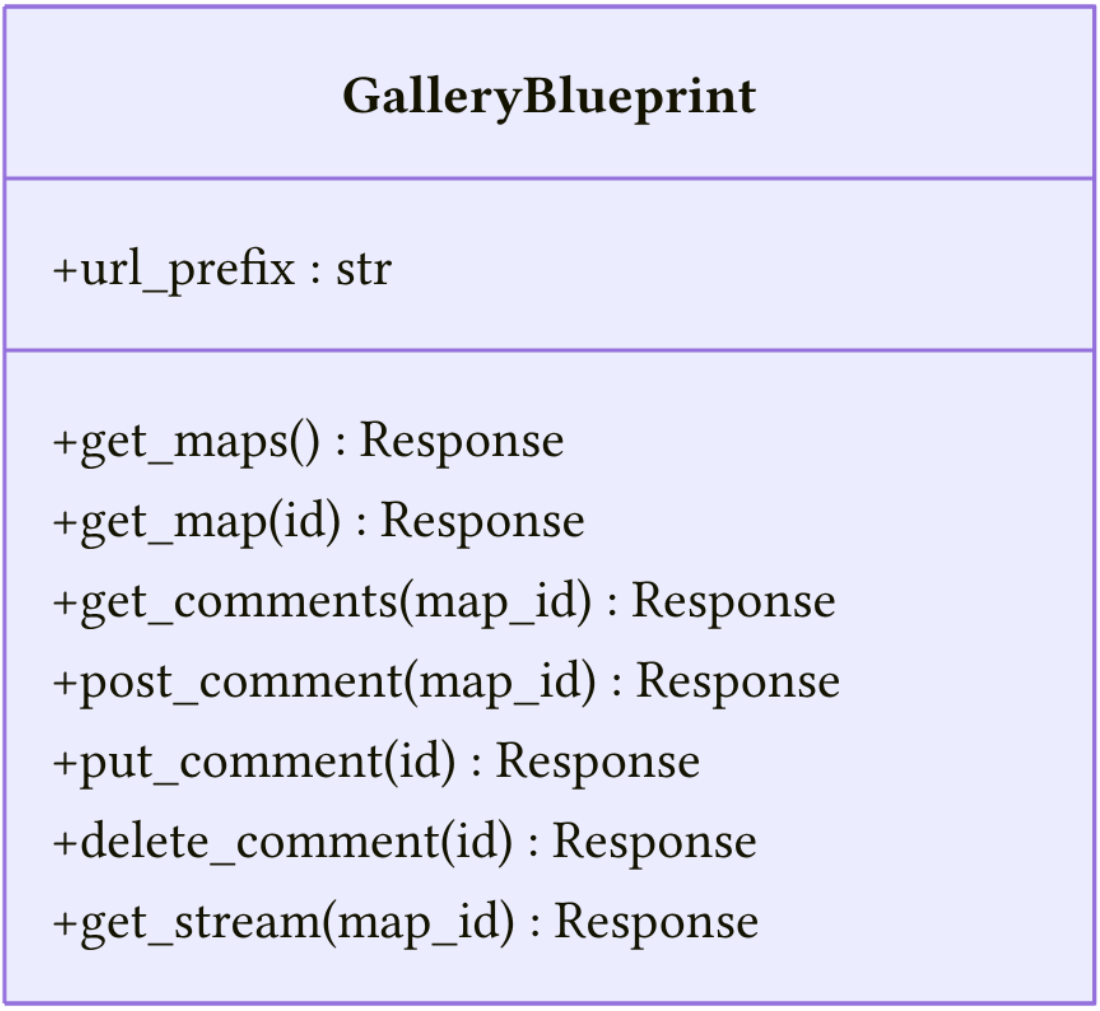


Рис. 13 — Слой 5 — API-маршруты